

CSC 769 Project 1

By Jiaxing Rong

I first created a struct called Node that includes the vertex, weight of the edge and next pointer points to the next Node. This data structure is prepared for the later linked list representation of the graph.

```
/**
 * An Node in the graph
 * @param vertex the vertex connected by the edge
 * @param weight the weight of the edge
 * @param next pointer points to the next Node
 */
struct Node
{
    int vertex;
    int weight;
    Node *next;
};
```

Linked list representation of the graph

```
/**
 * Undirected weighted graph
 */
class Graph
{
private:
    int V;
    Node **head;

public:
    /**
     * @param verticesCount number of vertices in the graph
     */
```

```

Graph(int verticesCount)
{
    V = verticesCount;
    head = new Node *[V];

    for (int i = 0; i < V; i++)
    {
        head[i] = nullptr;
    }
}

```

An method of the class Graph is addEdge that add an edge with the given vertex, another vertex and the weight of the edge

```

/**
 * add an edge in the graph
 * @param u one vertex connected by the edge
 * @param v another vertex connected by the edge
 * @param weight the weight of the edge
 */
void addEdge(int u, int v, int weight)
{
    // add v to u's list
    Node *newNode = new Node{v, weight, head[u]};
    head[u] = newNode;

    // add u to v's list
    newNode = new Node{u, weight, head[v]};
    head[v] = newNode;
}

```

The method findClosenessCentrality. I planed to use Floyd-Warshall to calculate closeness centrality. Therefore, I need to create distance matrix to keep track of the shortest distance between 2 vertexes

```

/**
 * Find closeness centrality in the graph
 */
void findClosenessCentrality()

```

```

{
    const int INF = 999999;

    // Create distance matrix
    int **dist = new int *[V];

    for (int i = 0; i < V; i++)
    {
        dist[i] = new int[V];

        for (int j = 0; j < V; j++)
        {
            if (i == j) // itself vertex
                dist[i][j] = 0;
            else // assign to infinity for other nodes
                dist[i][j] = INF;
        }
    }

    // Copy adjacency list into distance matrix
    for (int i = 0; i < V; i++)
    {
        Node *current = head[i];

        while (current != nullptr)
        {
            // update distance if current is the adjacent vetex
            to i

            dist[i][current->vertex] = current->weight;
            current = current->next;
        }
    }
}

```

Floyd-Warshall algorithm

```

// Floyd-warshall
for (int k = 0; k < V; k++)
{
    for (int i = 0; i < V; i++)

```

```

        {
            for (int j = 0; j < v; j++)
            {
                if (dist[i][k] != INF && dist[k][j] != INF &&
dist[i][k] + dist[k][j] < dist[i][j])
                {
                    // |i, k| + |k, j| < |i, j|
                    // update the shortest distance between i
and j if k is the intermediate vertex
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}

```

Print out the final distance matrix

```

// print out the distance matrix
cout << "Final Distance Matrix: " << endl << endl;
cout << " ";
for (int i = 0; i < v; i++) {
    cout << i << " ";
}
cout << endl;
cout << " ";
for (int i = 0; i < 2*v; i++) {
    cout << "-";
}
cout << endl;
for (int i = 0; i < v; i++) {
    cout << i << "|";
    for (int j = 0; j < v; j++) {
        cout << dist[i][j] << " ";
    }
    cout << endl;
}
cout << endl;

```

Find closeness centrality with the formula $\frac{1}{sum}$

```
// Find vertex with smallest total distance
int centralVertex = -1;
int minimumSum = INF;

for (int i = 0; i < V; i++)
{
    int sum = 0;

    for (int j = 0; j < V; j++)
    {
        sum += dist[i][j];
    }

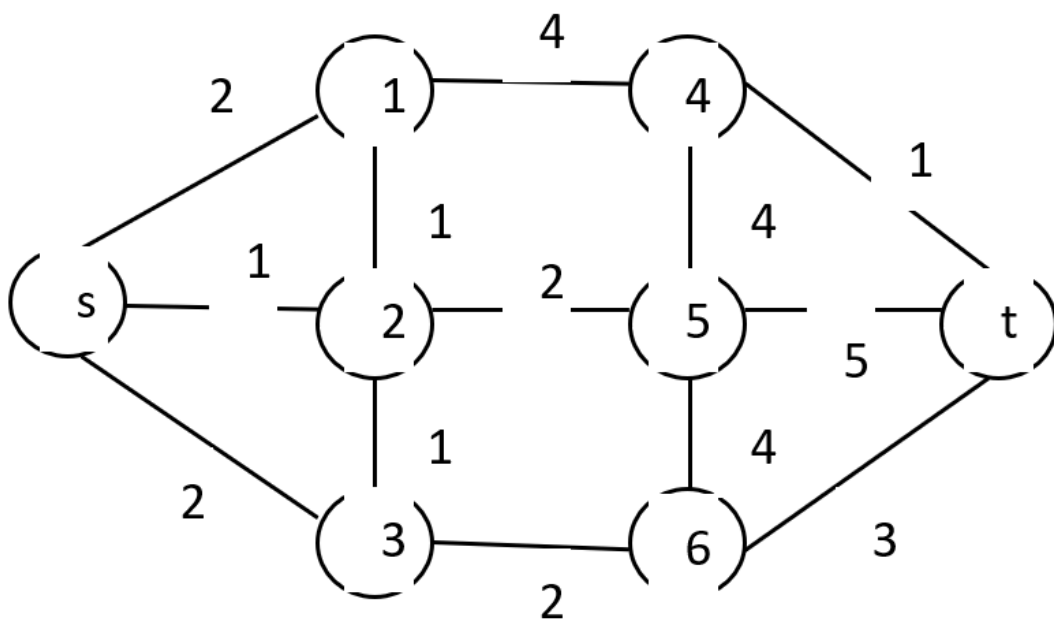
    double closeness =
        static_cast<double>(1) / sum;

    cout << "Vertex " << i << ": " << "sum of distance: "
    << sum << ", closeness = " << closeness << endl;

    if (sum < minimumSum)
    {
        minimumSum = sum;
        centralVertex = i;
    }
}

cout << "\nCloseness centrality vertex: " << centralVertex
<< endl;
```

Instantiate the below graph as input



In this case, I treated vertex s as 0 and t as 7

```
Graph g(8);  
// example input from the project document  
// treat vertex s as 0, vertex t as 7  
g.addEdge(0, 1, 2);  
g.addEdge(0, 2, 1);  
g.addEdge(0, 3, 2);  
g.addEdge(1, 2, 1);  
g.addEdge(1, 4, 4);  
g.addEdge(2, 3, 1);  
g.addEdge(2, 5, 2);  
g.addEdge(3, 6, 2);  
g.addEdge(4, 5, 4);  
g.addEdge(4, 7, 1);  
g.addEdge(5, 6, 4);  
g.addEdge(5, 7, 5);  
g.addEdge(6, 7, 3);  
g.findClosenessCentrality();
```

The output of the program

● Final Distance Matrix:

```
  0 1 2 3 4 5 6 7
-----
0|0 2 1 2 6 3 4 7
1|2 0 1 2 4 3 4 5
2|1 1 0 1 5 2 3 6
3|2 2 1 0 6 3 2 5
4|6 4 5 6 0 4 4 1
5|3 3 2 3 4 0 4 5
6|4 4 3 2 4 4 0 3
7|7 5 6 5 1 5 3 0
```

Vertex 0: sum of distance: 25, closeness = 0.04
Vertex 1: sum of distance: 21, closeness = 0.047619
Vertex 2: sum of distance: 19, closeness = 0.0526316
Vertex 3: sum of distance: 21, closeness = 0.047619
Vertex 4: sum of distance: 30, closeness = 0.0333333
Vertex 5: sum of distance: 24, closeness = 0.0416667
Vertex 6: sum of distance: 24, closeness = 0.0416667
Vertex 7: sum of distance: 32, closeness = 0.03125

Closeness centrality vertex: 2

Therefore, the closeness centrality is vertex 2 with the smallest sum distance between other vertexes